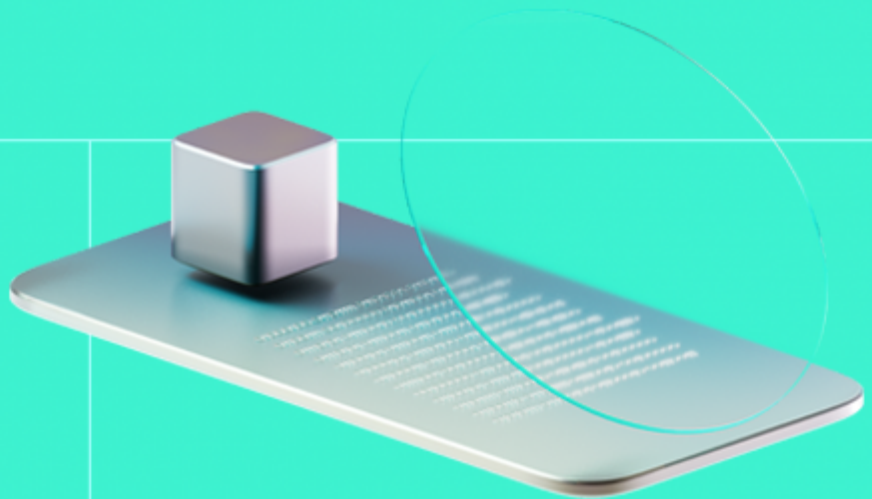




Smart Contract Code Review And Security Analysis Report

Customer: Darts

Date: 14/05/2026



We express our gratitude to the Darts team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The Darts implements an RWA investment protocol for Polygon Mainnet, where USDT-denominated products are registered, purchased, represented by ERC-721 investment NFTs, and later processed for periodic yield distribution and principal return. An ERC-7546 Meta Contract architecture is used to route investment proxy calls to modular function contracts, while Chainlink Automation-compatible upkeep logic is used to identify scheduled yield and maturity workflows.

Document

Name	Smart Contract Code Review and Security Analysis Report for Darts
Audited By	Ivan Bondar, Kerem Solmaz
Approved By	Kornel Światłowski
Website	
Changelog	14/05/2026 - Preliminary Report
Platform	Polygon
Language	Solidity
Tags	RWA; ERC-721; Upgradable; Claims
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/DART-s-dev/rwa-investment-contracts
Commit	2448ddd



Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

21	0	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	4
Low	5

Vulnerability	Severity	Status
F-2026-16869 - Empty NFT Batch in distributeYield Causes Permanent Automation Denial-of-Service and Product Lifecycle Deadlock	Medium	Pending Fix
F-2026-16871 - Reverting USDT Transfer in Push-Payment Loop Permanently Blocks Product Distribution and Maturity	Medium	Pending Fix
F-2026-16873 - Append-Only Product Registry Causes Permanent Denial of Service on Chainlink Automation checkUpkeep	Medium	Pending Fix
F-2026-17023 - Unrestricted NFT Transferability Bypasses Tier-Based Access Control After Purchase	Medium	Pending Fix
F-2026-16865 - Missing Zero-Address Validation in Initialization and Admin Management Leads to Permanent Misconfiguration	Low	Pending Fix
F-2026-16866 - Inconsistent Integer Size in Admin Array Iteration Leads to Denial of Service	Low	Pending Fix
F-2026-16870 - Restored Product Funding Does Not Clear isInsufficientBalance Flag, Causing Automation to Remain Stalled	Low	Pending Fix
F-2026-16949 - Missing Constraint Between Distribution Schedule End and Maturity Date Leads to Unexpected Locking Schedule	Low	Pending Fix
F-2026-17029 - Flat Admin Whitelist Without Role Separation Grants Full Operational Power to Every Admin Key	Low	Pending Fix
F-2026-16872 - Missing Duplicate Check in initialize Allows Redundant Admin Entries	Info	Pending Fix
F-2026-16938 - Floating Pragma, Unpinned and Deprecated Dependencies	Info	Pending Fix



Vulnerability	Severity	Status
F-2026-16945 - Missing Chain-Specific Scoping for Forwarder Address Environment Variable Risks Cross-Chain Misconfiguration	Info	Pending Fix
F-2026-16946 - Duplicate Implementation Deployment for SetTier Contract	Info	Pending Fix
F-2026-16947 - Unnecessary Custom String Parser for Admin Addresses Can Be Replaced With Native Foundry Cheatcode	Info	Pending Fix
F-2026-16955 - Missing Tier Registry Validation In Product Registration Can Temporarily Block Investment	Info	Pending Fix
F-2026-16956 - Missing Constraints In Admin Deletion Allow Owner To Temporarily Block Operations	Info	Pending Fix
F-2026-17004 - setAllowedByTierAddress Does Not Filter Zero-Address or Duplicate Entries	Info	Pending Fix
F-2026-17008 - Dead owner != address(0) Check in distributeYield and maturity	Info	Pending Fix
F-2026-17010 - Initialize Implementation Contract Not Locked with _disableInitializers Allows Direct Initialization	Info	Pending Fix
F-2026-17027 - Missing Event Emissions On State-Changing Functions Reduce Off-Chain Observability	Info	Pending Fix
F-2026-17028 - Test Contracts In Production Code	Info	Pending Fix

Documentation quality

- Functional requirements are not detailed.
 - Project overview is not provided.
 - All roles in the system are partially described.
 - Use cases are not described.
- Technical description is not detailed.
 - Contract interactions are not described.
 - Run instructions are not provided.
 - Technical specification is not provided.
 - The NatSpec documentation is sufficient.

Code quality

- The development environment is configured.

Test coverage

Coverage cannot be measured because `forge coverage` fails to compile the project (stack-too-deep).

- Foundry covers deployment and core flows in `src` (unit/fuzz) and `test/investment` (12 end-to-end scenarios).
- Negative paths are tested with revert checks on access control, eligibility, timing, and limits.
- Multi-user flows include several investors per product, tier-gated invest/mint, and admin, Safe, and automation roles.

Table of Contents

System Overview	7
Privileged Roles	
Potential Risks	11
Findings	14
Vulnerability Details	14
Disclaimers	67
Appendix 1. Definitions	68
Severities	68
Potential Risks	68
Appendix 2. Scope	69
Appendix 3. Additional Valuables	71

System Overview

The project implements an RWA investment protocol for Polygon Mainnet and Amoy, where USDT-denominated products are registered, purchased, represented by ERC-721 investment NFTs, and later processed for periodic yield distribution and principal return. An ERC-7546 Meta Contract architecture is used to route investment proxy calls to modular function contracts, while Chainlink Automation-compatible upkeep logic is used to identify scheduled yield and maturity workflows.

The investment system is structured around an ERC-7546 proxy-style bundle in which individual function contracts implement product registration, investing, admin NFT minting, pool deposits and withdrawals, yield distribution, maturity processing, tier configuration, admin control, and read-only getters. Shared state is accessed through ERC-7201-style namespaced storage libraries containing product records, admin allowlists, configuration values, and tier eligibility registries.

Products are registered by whitelisted administrators, and each product receives a deterministic **InvestmentNFT** contract deployed with `CREATE2`. Investors purchase units with USDT through `invest`, while administrators can mint NFTs through `mintNFT` for off-chain investment flows. Product state tracks offering limits, raised amount, product pool balance, schedule dates, expected yield, distribution progress, maturity progress, required membership tier, and insufficient-balance status.

Yield distribution and maturity are batch-oriented and process up to 50 NFT records per call through **InvestmentNFT**. Yield amounts are calculated from raised amount, expected yield, and elapsed distribution period, while maturity returns each NFT holder's recorded investment amount and burns the NFT. **Automation** scans all products through the investment interface and, when invoked by the configured Chainlink Forwarder, calls `distributeYield` or `maturity`; those investment calls remain subject to the investment admin allowlist because both functions use `onlyWhiteLists`.

Files in Scope

- **Getter.sol** — Read-only accessors and simulation helpers are provided for products, admins, tier registries, distribution dates, total yield, and per-NFT yield.
- **DistributionDateLib.sol** — Month-based distribution date calculations, midnight normalization, month-end alignment, and distribution period calculations are centralized.
- **PurchasePermissionLib.sol** — Tier eligibility is checked against registered ERC-721 and ERC-1155 SBT contracts before investment or admin NFT minting.
- **Initialize.sol** — Initial admin addresses, USDT address, and Safe multisig wallet address are written during one-time initialization.
- **ControlAdmin.sol** — Safe multisig-restricted admin list management is implemented through `addAdmin` and `deleteAdmin`.
- **InvestmentFacade.sol** — The facade exposes the external investment function surface used by the Meta Contract bundle.
- **CalculateYieldLib.sol** — Period yield, individual yield share, and batch tolerance calculations are provided for distribution logic and simulations.
- **SetTier.sol** — Whitelisted administrators can configure eligible SBT contract addresses and ERC-1155 token IDs per tier.
- **OnlyOwnerBase.sol** — The `onlyOwner` modifier is implemented by requiring the configured Safe multisig wallet as caller.

- **InvestmentNFT.sol** — The ERC-721-compatible investment NFT interface defines minting, burning, NFT information lookup, investment amount lookup, events, and errors.
- **IAutomation.sol** — Automation action types and the forwarder authorization error are defined.
- **InvestmentNFT.sol** — ERC-721 investment units are minted and burned by the investment proxy, with per-token investment amounts and admin-controlled metadata URI updates.
- **Deposit.sol** — Whitelisted administrators can deposit USDT into a product pool after product, amount, balance, allowance, and maturity checks.
- **DistributeYield.sol** — Whitelisted administrators can distribute periodic USDT yield to NFT holders in batches when scheduled distribution conditions are met.
- **InvestmentEvents.sol** — Product, investment, admin, distribution, maturity, and tier registry events are defined.
- **InvestmentErrors.sol** — Custom errors are defined for initialization, access control, product validation, funds, distribution, maturity, and tier configuration failures.
- **Withdraw.sol** — Whitelisted administrators can withdraw USDT from a product pool to the configured Safe multisig wallet.
- **Invest.sol** — Investors can purchase product units with USDT, subject to offering window, capacity, allowance, balance, and tier eligibility checks.
- **OnlyWhiteListsBase.sol** — The `onlyWhiteLists` modifier is implemented by checking the caller against the stored admin list.
- **MintNFT.sol** — Whitelisted administrators can mint investment NFTs for investors before operation start, supporting off-chain investment flows.
- **Automation.sol** — Chainlink Automation-compatible upkeep logic scans products and performs yield distribution or maturity calls through the investment interface when called by the configured forwarder.
- **Schema.sol** — Product, tier registry, admin list, configuration, and product registration argument structs are defined.
- **InvestmentFunctions.sol** — The investment function interface aggregates external state-changing and read-only function signatures.
- **Maturity.sol** — Whitelisted administrators can return principal to NFT holders, burn NFTs, and mark products mature in batches after maturity and distribution completion checks.
- **RegisterProduct.sol** — Whitelisted administrators can validate product parameters, deploy a deterministic **InvestmentNFT**, and store product configuration.
- **Storage.sol** — ERC-7201-style storage slots and accessors are defined for product, whitelist, configuration, and tier registry state.
- **Investment.sol** — Investment functions, errors, and events are combined into a single protocol interface.
- **InvestmentDeployer.sol** — The Meta Contract bundle is assembled by registering function selectors, deploying the facade, and initializing the investment proxy.
- **DeployAutomation.s.sol** — The Chainlink Automation-compatible contract is deployed from environment-provided investment proxy and forwarder addresses.
- **DeployInvestment.s.sol** — The investment proxy is deployed from environment configuration, and dictionary ownership is transferred to the configured Safe multisig wallet.

Privileged roles

Dictionary (ERC-7546 / MC framework)

- **Dictionary Owner** (`SAFE_MULTISIG_WALLET` after deployment): Function-level upgrade authority over all proxy-dispatched code.

- Can call `setImplementation` to replace any registered function selector's implementation contract, including fund-moving functions such as `withdraw`, `distributeYield`, and `maturity`. Replacement is instant and silent - no timelock, multi-step confirmation, or on-chain delay is enforced by the Dictionary contract.
- Can call `upgradeFacade` to replace the ABI surface contract returned by the proxy's `implementation` view.
- Can call `transferOwnership` to transfer Dictionary ownership to another address.
- Can call `renounceOwnership` to permanently disable all future function upgrades and facade changes.

ControlAdmin.sol

- **SAFE_MULTISIG_WALLET** (inherited from **OnlyOwnerBase**): Admin membership can be managed.
 - Can call `addAdmin` to add an address to the admin list.
 - Can call `deleteAdmin` to remove an address from the admin list.

SetTier.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Tier-based purchase eligibility configuration can be managed.
 - Can call `setAllowedByTierAddress` to replace the SBT contract allowlist for a tier and clear removed ERC-1155 token ID bindings.
 - Can call `setAllowedByTierId` to replace ERC-1155 token IDs for a registered SBT contract under a tier.

InvestmentNFT.sol

- **Investment admin**: NFT metadata can be updated through the admin list stored in the investment contract.
 - Can call `setURI` to update the shared metadata URI returned for all token IDs.
- **Owner**: Investment NFTs can be minted and burned.
 - Can call `mint` to mint an NFT and record its investment amount.
 - Can call `burn` to burn an NFT and delete its recorded investment amount.

Deposit.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Product pool funding can be supplied.
 - Can call `deposit` to transfer USDT into a product pool.

DistributeYield.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Yield distribution can be executed.
 - Can call `distributeYield` to distribute product yield to NFT holders in batches.

Withdraw.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Product pool funds can be withdrawn to the Safe multisig wallet.
 - Can call `withdraw` to transfer USDT from a product pool to `SAFE_MULTISIG_WALLET`.

Invest.sol

- **Eligible investor:** Product purchases can be made when the required tier permission is satisfied.
 - Can call `invest` to purchase product units, transfer USDT, increase product accounting, and mint an investment NFT.

MintNFT.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Off-chain investments can be represented with NFTs.
 - Can call `mintNFT` to mint an investment NFT for an eligible investor and increase product raised amount.

Automation.sol

- **Configured forwarder:** Automated product lifecycle actions can be executed through the automation contract.
 - Can call `performUpkeep` to trigger `distributeYield` or `maturity` on the investment contract for the encoded product and action type.

Maturity.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Product maturity processing can be executed.
 - Can call `maturity` to return principal to NFT holders, burn NFTs, and mark product maturity progress.

RegisterProduct.sol

- **Admin** (inherited from **OnlyWhiteListsBase**): Investment products can be created.
 - Can call `registerProduct` to validate product parameters, deploy the product NFT contract, and store product configuration.

Potential Risks

Out-of-Scope Components and Third-Party Dependencies

- **Dependence on MC proxy and dictionary logic:** The investment bundle is deployed through **InvestmentDeployer** with `MCDevKit`, selector-to-implementation mappings are registered through `mc.use`, and the resulting proxy address is produced by `mc.deploy`. The deployed system therefore depends on the MC ERC-7546 proxy and dictionary implementation imported from external packages, while **DeployInvestmentScript** transfers **Dictionary** ownership to the configured Safe multisig. Incorrect selector routing, dictionary behavior, or upgrade ownership handling in that external framework could affect every function contract exposed through **InvestmentFacade**.
- **Dependence on configured external asset and eligibility contracts:** Core accounting and settlement are performed against the ERC-20 address stored as `USDT_ADDRESS` during `initialize`, and `invest`, `deposit`, `withdraw`, `distributeYield`, and `maturity` all transfer that external token. The protocol is hardcoded to a single payment token (USDT on Polygon) and does not support rebasing, fee-on-transfer, or other non-standard ERC-20 token mechanics - all internal accounting credits the nominal transfer amount without balance-delta verification. Investor eligibility is delegated to external ERC-721 or ERC-1155 contracts registered through `setAllowedByTierAddress` and `setAllowedByTierId`, which are queried by **PurchasePermissionLib** through `supportsInterface`, `balanceOf`, and token-id balances. Unexpected behavior or misconfiguration of these external token contracts could block investments, distributions, redemptions, or tier-gated access.

Loop Efficiency

- **Product scanning grows with registered products:** **Automation** calls `getAllProducts` and scans every product returned from `productIdKeys` inside `checkUpkeep`. Each call copies all stored **Schema.Product** records into memory before a required distribution or maturity action is selected. A growing product registry could make upkeep checks increasingly expensive and could prevent automated processing from being discovered within practical gas limits.
- **Batch settlement depends on repeated external calls:** **DistributeYield** and **Maturity** process NFT holders in batches returned by `getNFTInfos`, which are capped to a 50-token window. Each distribution batch performs USDT transfers to NFT holders, and each maturity batch performs USDT transfers and burns NFTs through **InvestmentNFT**. Distribution and redemption of a large product therefore require repeated upkeep or admin-triggered calls across sequential token ID batches, and later holders depend on successful completion of earlier batches.

Cross-Product Fund Isolation

- **All products share a single USDT balance with soft per-product accounting:** Investor USDT from `invest`, admin USDT from `deposit`, yield payouts via `distributeYield`, principal returns via `maturity`, and admin withdrawals via `withdraw` all operate on the same `usdt.balanceOf(address(this))`. The `productPool` field per product is an internal accounting counter with no actual token segregation. If cumulative rounding from yield calculations across many distributions causes the sum of all `productPool` values to drift from the real contract balance, one product's settlement could fail because another product's funds are masking the deficit. The contract checks both

`productPool` and `usdt.balanceOf(address(this))` independently, but no invariant enforcement ensures their aggregate consistency.

Permissions, Authorization, and Access

- **Operational powers are concentrated in whitelisted admins:** The `onlyWhiteLists` modifier gates `registerProduct`, `mintNFT`, `deposit`, `withdraw`, `distributeYield`, `maturity`, `setAllowedByTierAddress`, and `setAllowedByTierId` through the admin array stored in **WhiteListsState**. These functions cover product creation, off-chain investment NFT issuance, product pool funding, product pool withdrawal, investor eligibility configuration, yield distribution, and maturity redemption. Compromise or misconfiguration of an admin address could affect product terms, eligibility, settlement timing, and protocol-held USDT flows.
- **Automation execution depends on admin-list configuration:** **Automation** restricts `performUpkeep` to the configured `forwarderAddress`, but the subsequent calls to `distributeYield` and `maturity` are made from the **Automation** contract to functions protected by `onlyWhiteLists`. Automated settlement can therefore execute only if the **Automation** contract address is also present in the investment admin list. A mismatch between the configured forwarder, automation address, and admin list could prevent scheduled actions from being performed through the automated path.
- **Treasury movements are executable by whitelisted admins:** **Withdraw** allows any whitelisted admin to reduce a product's `productPool` and transfer USDT from the investment proxy to `SAFE_MULTISIG_WALLET`. The same admin role can call `deposit` to increase product liquidity and `distributeYield` or `maturity` to move USDT to NFT holders. Product pool balances and settlement liquidity are therefore dependent on admin-operated fund movements rather than an autonomous reserve model.
- **Tier access is controlled by admin-managed external token lists:** **SetTier** replaces the full SBT contract list for each tier and sets ERC-1155 token IDs for registered SBT contracts, while **PurchasePermissionLib** grants purchase permission based on balances in those configured contracts. Product registration stores only a numeric `requiredTier`, and the meaning of that tier is determined later by the admin-managed registry. Changes to tier contract lists or token IDs can alter who is eligible to call `invest` or receive admin-minted investment NFTs for existing products.
- **NFT metadata is mutable by investment admins:** **InvestmentNFT** allows `setURI` to be called by any address present in `getAdminList` on the investment contract. The same URI is returned for all token IDs in the product NFT contract. Admin-controlled metadata changes can alter the off-chain representation of investment NFTs after they have been minted and transferred.
- **No pause mechanism or circuit breaker exists:** The protocol does not implement a pause function or emergency stop across any of its function contracts. If a vulnerability is discovered during active product operation, there is no way to freeze investments, distributions, withdrawals, or maturity processing short of upgrading the affected function implementation through the **Dictionary** (which requires multisig action but executes instantly once approved). All functions remain callable during any incident window between detection and remediation.
- **No early redemption or refund path exists before maturity:** Once an investor calls `invest` or receives an NFT via `mintNFT`, there is no on-chain mechanism to exit the position before the product reaches maturity. No burn-and-refund function, no early withdrawal, and no partial redemption are available. The investor's only exit path is transferring the **InvestmentNFT** to another party through standard ERC-721 transfer. If the underlying RWA underperforms, if the investor requires liquidity, or if the product experiences extended delays due to insufficient pool balances, the investor remains locked with no protocol-level recourse until `maturity` executes.

Centralization

- **Single configured Safe controls admin membership and dictionary ownership:** **OnlyOwnerBase** treats `SAFE_MULTISIG_WALLET` as the owner for `addAdmin` and `deleteAdmin`, and **DeployInvestmentScript** transfers **Dictionary** ownership to the same configured address. This address controls the admin set used by operational function contracts and the ownership position for selector-to-implementation routing. Loss of control over that address could affect both privileged operations and future function routing management.

Upgradeability

- **Dictionary upgrades are instant with no timelock or delay:** The **Dictionary** contract's `setImplementation` function (inherited from `DictionaryBase`) directly overwrites the `functions[selector]` mapping for any registered selector in a single transaction. No timelock, governance vote, multi-step confirmation, or on-chain delay is enforced. The Dictionary owner (`SAFE_MULTISIG_WALLET` after deployment) can replace the implementation behind any of the 22 registered selectors — including `withdraw`, `distributeYield`, `maturity`, and `initialize` — with immediate effect on all subsequent proxy calls.
- **Function-level upgrades affect shared namespaced storage:** The deployed investment system is assembled from separate function contracts mapped through the MC dictionary, while all implementation contracts read and write shared ERC-7201 storage slots in **Storage**. **Schema.Product**, **WhiteListsState**, **ConfigState**, and **TierRegistryState** are relied on across independent functions such as `registerProduct`, `invest`, `distributeYield`, `maturity`, and `withdraw`. An incompatible function implementation upgrade could reinterpret or modify shared product, admin, config, or tier state used by other selectors.
- **Active products can span upgrade periods:** Product state includes multi-step fields such as `distributedCount`, `distributedTokenId`, `distributedYieldPerCount`, `totalReturnedAmount`, `maturedTokenId`, and `isInsufficientBalance`. **DistributeYield** and **Maturity** use these fields to resume batched processing across multiple calls, while the MC dictionary model permits individual function implementations to be changed after deployment. Upgrades applied while a product is mid-distribution or mid-maturity could affect pending batches and settlement continuity.

Off-Chain Dependency and Liveness

- **Automated settlement depends on Chainlink Forwarder execution:** **Automation** restricts `performUpkeep` to the configured `forwarderAddress` and uses that call path to invoke `distributeYield` or `maturity` on the investment contract. Although the underlying functions remain admin-callable through the investment proxy, the automated path depends on Chainlink Automation checks, the configured forwarder address, and the automation contract being authorized as an investment admin. Forwarder downtime, incorrect deployment configuration, or missed upkeep execution could delay scheduled yield distribution and maturity processing.
- **Insufficient product pools halt automated processing until replenished and retried:** **DistributeYield** and **Maturity** set `isInsufficientBalance` and return when `productPool` cannot cover the remaining yield or principal obligation. **Automation** explicitly skips products with `isInsufficientBalance` in `checkUpkeep`, and `deposit` does not clear that flag. A product with insufficient pool accounting can therefore stop being selected by automation until an admin deposits enough USDT and successfully retries the relevant distribution or maturity path.

Findings

Vulnerability Details

F-2026-16869 - Empty NFT Batch in distributeYield Causes Permanent Automation Denial-of-Service and Product Lifecycle Deadlock - Medium

Description:

When a product is registered but no investor subscribes before the first distribution date (`product.raisedAmount == 0`), the Chainlink Automation keeper enters an infinite loop on that product. The `distributeYield` function computes `startTokenId = distributedTokenId + 1 = 1` and queries the NFT contract for a batch starting at token ID 1. Because no NFTs were minted, `tokenIdCounter` is 0. In `InvestmentNFT.getNFTInfos()`, the batch-fetching logic returns an empty array when `startTokenId` exceeds `tokenIdCounter`:

```
// InvestmentNFT.getNFTInfos()
uint256 endTokenId = startTokenId + 49 < tokenIdCounter
    ? startTokenId + 49
    : tokenIdCounter;
uint256 length = endTokenId >= startTokenId
    ? endTokenId - startTokenId + 1
    : 0;
```

With `startTokenId = 1` and `tokenIdCounter = 0`, `endTokenId` resolves to 0 and `length` to 0. The distribution loop executes zero iterations, so `distributedCount` is never incremented, `distributedTokenId` is never advanced, and `isInsufficientBalance` is set to false. The call succeeds as a silent no-op. On the next upkeep cycle, `checkUpkeep` sees `distributedCount < totalDistributionCount` and the due date still satisfied, so it returns the same product again. All later products in the scan array are never reached, blocking their automated settlement indefinitely.

The stuck product itself can never mature because `maturity` reverts with `BeforeDistributionCompleted` while `distributedCount` remains below `totalDistributionCount`. Investors holding NFTs in later products experience indefinite settlement delay unless operators manually call settlement functions to bypass the Automation pipeline.

Assets:

- `DistributeYield.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

- Maturity.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- InvestmentNFT.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Automation.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Medium

Severity: Medium

Recommendations

Remediation:

Add an explicit guard for zero-liability products. If `product.raisedAmount == 0` at distribution time, no investor subscribed and the distribution round should be finalized immediately:

```
if (product.raisedAmount == 0) {
    product.distributedCount++;
    product.distributedYieldPerCount = 0;
    product.isInsufficientBalance = false;
    emit IInvestmentEvents.YieldDistributed(productId, product.distributedCount, 0, 0, 0);
    return;
}
```

Apply an equivalent guard in `maturity`. Alternatively, make `checkUpkeep` skip products where `raisedAmount == 0`. Checking `raisedAmount` is preferred over `nft.tokenIdCounter()` because it is the business-level invariant (no capital raised means no investor liability) and avoids an external call to the NFT contract.

Evidences

Empty NFT Batch in distributeYield Causes Permanent Automation Denial-of-Service and Product Lifecycle Deadlock

Reproduce:

PoC Steps:

- Deploy the full Investment system (proxy + Automation + mock USDT). Register Product 1 (no investors) and Product 2 (with an investor holding 4 units and a funded pool).
- Warp time to the distribution date. Confirm Product 1's NFT contract has `tokenIdCounter == 0`.
- Call `distributeYield(PRODUCT_1)` - succeeds but `distributedCount` stays at 0 (silent no-op).
- Call `checkUpkeep` three times, executing `performUpkeep` between each - every call returns Product 1. Product 2 is never reached (infinite loop).
- Call `maturity(PRODUCT_1)` - reverts with `BeforeDistributionCompleted` because `distributedCount` can never reach `totalDistributionCount` (permanent deadlock).

PoC Code:

```
function test_EmptyNFTBatchDoS() public {
    // Register product 1 – no one will invest
    vm.prank(admin1);
    investment.registerProduct(Schema.RegisterProductArgs({
        productId: PRODUCT_1,
        offeringAmount: OFFERING_AMOUNT,
        minInvestment: MIN_INVESTMENT,
        offeringEndDate: OFFERING_END_DATE,
        maturityDate: MATURITY_DATE,
        expectedYield: EXPECTED_YIELD,
        operationStartDate: OPERATION_START_DATE,
        distributionStartDate: MATURITY_DATE,
        baseTokenURI: "https://example.com/1/",
        totalDistributionCount: 1,
        distributionInterval: 0,
        requiredTier: 0
    }));

    // Register product 2 – has real investors
    vm.prank(admin1);
    investment.registerProduct(Schema.RegisterProductArgs({
        productId: PRODUCT_2,
        offeringAmount: OFFERING_AMOUNT,
        minInvestment: MIN_INVESTMENT,
        offeringEndDate: OFFERING_END_DATE,
        maturityDate: MATURITY_DATE,
        expectedYield: EXPECTED_YIELD,
        operationStartDate: OPERATION_START_DATE,
        distributionStartDate: MATURITY_DATE,
        baseTokenURI: "https://example.com/2/",
```



```

        totalDistributionCount: 1,
        distributionInterval: 0,
        requiredTier: 0
    }));

    // Invest in product 2 (product 1 stays at zero NFTs)
    vm.prank(admin1);
    investment.mintNFT(PRODUCT_2, 4, investor1);

    // Fund product 2's pool for yield + principal
    Schema.Product memory p2 = investment.getProduct(PRODUCT_2);
    uint256 depositAmount = p2.raisedAmount * 2;
    usdt.mint(admin1, depositAmount);
    vm.startPrank(admin1);
    usdt.approve(address(investment), depositAmount);
    investment.deposit(PRODUCT_2, depositAmount);
    vm.stopPrank();

    // Warp past the distribution/maturity date
    vm.warp(MATURITY_DATE);

    // Confirm product 1's NFT contract has zero minted tokens
    Schema.Product memory p1 = investment.getProduct(PRODUCT_1);
    assertEq(InvestmentNFT(p1.nftContract).tokenIdCounter(), 0, "product 1 has zero NFTs");

    assertEq(p1.distributedCount, 0);

    vm.prank(admin1);
    investment.distributeYield(PRODUCT_1);

    Schema.Product memory p1After = investment.getProduct(PRODUCT_1);
    assertEq(p1After.distributedCount, 0, "distributeYield was a no-op: distributedCount still 0");

    (bool needed1, bytes memory data1) = automation.checkUpkeep(abi.encode(0));
    assertTrue(needed1);
    (, uint256 returnedId1) = abi.decode(data1, (IAutomation.ActionType, uint256));
    assertEq(returnedId1, PRODUCT_1, "1st checkUpkeep returns stuck product 1");

    vm.prank(forwarder);
    automation.performUpkeep(data1);

    (bool needed2, bytes memory data2) = automation.checkUpkeep(abi.encode(0));
    assertTrue(needed2);

```

```

        (, uint256 returnedId2) = abi.decode(data2, (IAutomation.ActionType, uint
256));

        assertEq(returnedId2, PRODUCT_1, "2nd checkUpkeep still returns product 1
, product 2 starved");

        vm.prank(forwarder);
        automation.performUpkeep(data2);
        (bool needed3, bytes memory data3) = automation.checkUpkeep(abi.encode(0)
);
        assertTrue(needed3);
        (, uint256 returnedId3) = abi.decode(data3, (IAutomation.ActionType, uint
256));
        assertEq(returnedId3, PRODUCT_1, "3rd checkUpkeep still returns product 1
, infinite loop");

        vm.expectRevert(IInvestmentErrors.BeforeDistributionCompleted.selector);
        vm.prank(admin1);
        investment.maturity(PRODUCT_1);
    }

```

Results:

```

Ran 1 test for test/PoC_EmptyNFTBatchDoS.t.sol:PoC_EmptyNFTBatchDoS
[PASS] test_EmptyNFTBatchDoS() (gas: 4841714)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 51.61ms (3.62ms
CPU time)

```

Files:

PoC_EmptyNFTBatchDoS.t.sol

F-2026-16871 - Reverting USDT Transfer in Push-Payment Loop Permanently Blocks Product Distribution and Maturity - Medium

Description:

The `distributeYield` and `maturity` functions iterate over a batch of up to 50 NFT holders and call `safeTransfer` for each recipient within the same transaction. If any single transfer reverts, the entire transaction reverts and the batch cursor (`distributedTokenId` / `maturedTokenId`) is never advanced:

```
for (uint256 i = 0; i < nftInfos.length; i++) {  
    if (nftInfos[i].owner != address(0)) {  
        usdt.safeTransfer(nftInfos[i].owner, individualPeriodYield);  
        // ...  
    }  
}
```

USDT on Ethereum, Polygon, and other chains includes a blacklist mechanism. If any NFT holder's address is blacklisted by Tether, `transfer` to that address reverts. Because `SafeERC20.safeTransfer` propagates the revert, the batch containing the blacklisted holder becomes permanently stuck. Every retry hits the same revert at the same token ID.

Since `distributedCount` never reaches `totalDistributionCount`, the `maturity` function also reverts with `BeforeDistributionCompleted`, locking all investors' principal and yield for the affected product. The blacklisted holder could transfer their NFT to a clean address to unblock the batch, but this requires active cooperation from an account that may have restricted transactability.

Assets:

- `DistributeYield.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Maturity.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Medium

Recommendations

Remediation:

Wrap individual `safeTransfer` calls in a try/catch. On failure, escrow the failed amount in a pull-based mapping and allow holders to claim later:

```
try usdt.transfer(nftInfos[i].owner, individualPeriodYield) returns (bool success) {
    if (!success) {
        unclaimedYield[nftInfos[i].owner][productId] += individualPeriodYield
    }
} catch {
    unclaimedYield[nftInfos[i].owner][productId] += individualPeriodYield;
}
```

Evidences

Reverting USDT Transfer in Push-Payment Loop Permanently Blocks Product Distribution and Maturity

Reproduce:

PoC Steps:

- Register a product, mint NFTs for two investors, and fund the pool via admin deposit.
- Warp to the distribution date. Simulate USDT blacklisting `investor2` using `vm.mockCallRevert` on `transfer(investor2, ...)`.
- Call `distributeYield` - reverts because `safeTransfer` to the blacklisted holder fails, rolling back the entire batch.
- Retry `distributeYield` - same revert at the same token ID, `distributedCount` still 0.
- Call `maturity` - reverts with `BeforeDistributionCompleted` since `distributedCount` never reached `totalDistributionCount`. All investors' principal and yield are locked.

PoC Code:

```
function test_BlacklistedHolderBlocksDistribution() public {
    vm.prank(admin1);
    investment.registerProduct(Schema.RegisterProductArgs({
        productId: PRODUCT_1,
        offeringAmount: OFFERING_AMOUNT,
        minInvestment: MIN_INVESTMENT,
        offeringEndDate: OFFERING_END_DATE,
```

```

        maturityDate: MATURITY_DATE,
        expectedYield: EXPECTED_YIELD,
        operationStartDate: OPERATION_START_DATE,
        distributionStartDate: MATURITY_DATE,
        baseTokenURI: "https://example.com/1/",
        totalDistributionCount: 1,
        distributionInterval: 0,
        requiredTier: 0
    }));

    vm.startPrank(admin1);
    investment.mintNFT(PRODUCT_1, 2, investor1);
    investment.mintNFT(PRODUCT_1, 2, investor2);
    vm.stopPrank();

    Schema.Product memory p = investment.getProduct(PRODUCT_1);
    uint256 depositAmount = p.raisedAmount * 2;
    usdt.mint(admin1, depositAmount);
    vm.startPrank(admin1);
    usdt.approve(address(investment), depositAmount);
    investment.deposit(PRODUCT_1, depositAmount);
    vm.stopPrank();

    vm.warp(MATURITY_DATE);

    // Simulate USDT blacklisting investor2
    vm.mockCallRevert(
        address(usdt),
        abi.encodeWithSelector(IERC20.transfer.selector, investor2),
        "Blacklisted"
    );

    // First attempt reverts on the blacklisted holder
    vm.expectRevert();
    vm.prank(admin1);
    investment.distributeYield(PRODUCT_1);

    p = investment.getProduct(PRODUCT_1);
    assertEq(p.distributedCount, 0, "distribution stuck after 1st attempt");

    // Retry hits the same revert at the same token ID
    vm.expectRevert();
    vm.prank(admin1);
    investment.distributeYield(PRODUCT_1);

    p = investment.getProduct(PRODUCT_1);
    assertEq(p.distributedCount, 0, "distribution stuck after 2nd attempt");

    // Maturity blocked because distributedCount < totalDistributionCount

```

```
vm.expectRevert(IIInvestmentErrors.BeforeDistributionCompleted.selector);  
vm.prank(admin1);  
investment.maturity(PRODUCT_1);  
}
```

Results:

```
Ran 1 test for test/PoC_BlacklistBlocksSettlement.t.sol:PoC_BlacklistBlocksSe  
ttlement  
[PASS] test_BlacklistedHolderBlocksDistribution() (gas: 2677518)  
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 51.16ms (2.19ms  
CPU time)
```

Files:

PoC_BlacklistBlocksSettlement.t.sol

F-2026-16873 - Append-Only Product Registry Causes Permanent Denial of Service on Chainlink Automation checkUpkeep - Medium

Description:

The `productIdKeys` array is append-only; no function removes entries after products mature. The `getAllProducts` function allocates a memory array for the entire registry and copies every product record:

```
uint256 productCount = Storage.ProductsState().productIdKeys.length;
productsInfo = new Schema.Product[](productCount);
for (uint256 i = 0; i < productCount; i++) {
    uint256 pid = Storage.ProductsState().productIdKeys[i];
    productsInfo[i] = Storage.ProductsState().products[pid];
}
```

In **Automation**, `checkUpkeep` calls `getAllProducts` via `STATICCALL` to the proxy and then iterates the full result, including `calculateNextDistributionDate` computation per non-matured entry. Per the Chainlink Automation documentation, the `checkGasLimit` on Polygon Mainnet is 10,000,000. Once approximately 170-200 lifetime products accumulate, `checkUpkeep` exceeds this hard limit and reverts.

Chainlink nodes will stop performing upkeep entirely, permanently disabling automated yield distribution and maturity processing for all active products simultaneously. No admin function exists to prune matured entries from the array, so no recovery path is available without contract redeployment. Manual calls to `distributeYield` and `maturity` remain functional but negate the purpose of the Chainlink integration.

Assets:

- Getter.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- RegisterProduct.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Automation.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: Maintain a separate array of active (non-matured) product IDs. Remove entries from the active set when `isMaturity` is set to `true`. Modify `checkUpkeep` to iterate only over the active array. Alternatively, implement pagination in `checkUpkeep` using an offset parameter so the keeper can scan in segments across multiple upkeep cycles.

Evidences

Append-Only Product Registry Causes Permanent Denial of Service on Chainlink Automation checkUpkeep

Reproduce:

PoC Steps:

- Register 50 products.
- Assert `getAllProducts().length == 50`. Measure `checkUpkeep` gas via `gasleft()`.
- Register 50 more products (100 total). Assert `getAllProducts().length == 100` — no pruning mechanism exists.
- Measure `checkUpkeep` gas again. Assert it grew compared to the 50-product measurement.
- Compute marginal warm-storage gas per product. Adjust to cold-storage estimate (+23 SLOADs * 2000 gas delta). Assert that fewer than 300 lifetime products breach Chainlink's 10M `checkGasLimit`.

PoC Code:

```
function test_AppendOnlyRegistryGasGrowth() public {
    uint256 batch = 50;

    for (uint256 i = 1; i <= batch; i++) {
        vm.prank(admin1);
        investment.registerProduct(_args(i));
    }
    assertEq(investment.getAllProducts().length, batch);

    uint256 g1 = gasleft();
    automation.checkUpkeep(abi.encode(0));
    uint256 gasAfter50 = g1 - gasleft();

    for (uint256 i = batch + 1; i <= batch * 2; i++) {
```

```

        vm.prank(admin1);
        investment.registerProduct(_args(i));
    }

    assertEq(investment.getAllProducts().length, batch * 2, "registry is append-only, no pruning");

    uint256 g2 = gasleft();
    automation.checkUpkeep(abi.encode(0));
    uint256 gasAfter100 = g2 - gasleft();

    assertGt(gasAfter100, gasAfter50, "checkUpkeep gas grows with product count");

    // Marginal gas per product (warm-storage lower bound; cold SLOADs cost 2
    1x more)
    uint256 warmGasPerProduct = (gasAfter100 - gasAfter50) / batch;

    // Each product struct requires ~23 SLOADs. In production each checkUpkeep
    p
    // is a fresh tx with cold storage (2100 gas/SLOAD vs 100 warm).
    // Adjust warm measurement to cold estimate.
    uint256 coldGasPerProduct = warmGasPerProduct + 23 * (2100 - 100);

    uint256 CHAINLINK_CHECK_GAS_LIMIT = 10_000_000;
    uint256 maxProducts = CHAINLINK_CHECK_GAS_LIMIT / coldGasPerProduct;

    assertLt(maxProducts, 300, "fewer than 300 lifetime products breach Chain
    link's 10M checkGasLimit");
}

```

Results:

```

Ran 1 test for test/PoC_AppendOnlyRegistryDoS.t.sol:PoC_AppendOnlyRegistryDoS
[PASS] test_AppendOnlyRegistryGasGrowth() (gas: 217157320)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 82.56ms (35.70ms
CPU time)

```

Files:

PoC_AppendOnlyRegistryDoS.t.sol

F-2026-17023 - Unrestricted NFT Transferability Bypasses Tier-Based Access Control After Purchase - Medium

Description:

Tier-based eligibility is enforced exclusively at the point of investment. In **Invest**, the `invest` function calls `hasPurchasePermission` to verify SBT ownership before allowing participation, and **MintNFT** applies the same check for admin-minted positions. After purchase, the **InvestmentNFT** contract inherits standard OpenZeppelin `ERC721` without overriding transfer hooks:

```
contract InvestmentNFT is IInvestmentNFT, ERC721, Ownable {  
    // No _update override, no transfer restriction  
}
```

The `distributeYield` and `maturity` functions pay yield and return principal to `nftInfos[i].owner`, which resolves to whoever currently holds the NFT at execution time. No eligibility re-check occurs at distribution or redemption. A non-qualified address that would fail the tier gate at purchase can acquire the NFT through a standard ERC-721 transfer and receive all future yield and principal payments.

Once a non-compliant address holds an **InvestmentNFT**, no admin function exists to revoke, freeze, or skip that holder during settlement. The only path to exclude such an address is a contract upgrade through the Dictionary, which requires multisig action and affects the implementation for all products simultaneously.

For regulated RWA products subject to investor accreditation, KYC, or jurisdictional requirements, the on-chain position can rotate to addresses that would not have passed the original eligibility gate, with no protocol-level mechanism to remediate the compliance violation without a code-level intervention.

Assets:

- `PurchasePermissionLib.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentNFT.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate:

3/5

Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Apply one of the following mitigations:

Option A: Make the InvestmentNFT non-transferable (soul-bound).

Override `_update` to block all transfers except mints and burns:

```
function _update(address to, uint256 tokenId, address auth) internal override
returns (address) {
    address from = _ownerOf(tokenId);
    if (from != address(0) && to != address(0)) {
        revert("InvestmentNFT: non-transferable");
    }
    return super._update(to, tokenId, auth);
}
```

Option B: Enforce tier eligibility on every transfer. Expose a purchase permission check on the Investment contract by registering a new external view function:

```
// New function in Investment scope (has access to PurchasePermissionLib and
Storage)
function hasPurchasePermission(address user, uint8 requiredTier) external view
returns (bool) {
    return PurchasePermissionLib.hasPurchasePermission(user, requiredTier);
}
```

Override `_update` in **InvestmentNFT** to call it before allowing transfers:

```
function _update(address to, uint256 tokenId, address auth) internal override
returns (address) {
    address from = _ownerOf(tokenId);
    if (from != address(0) && to != address(0)) {
        Schema.Product memory product = IInvestment(investmentContract).getPr
oduct(productId);
        if (product.requiredTier != 0) {
            if (!IInvestment(investmentContract).hasPurchasePermission(to, pr
oduct.requiredTier)) {
                revert NotEligible(product.requiredTier);
            }
        }
    }
    return super._update(to, tokenId, auth);
}
```

```

    }
  }
  return super._update(to, tokenId, auth);
}

```

Option C: Accept the design and document it. If unrestricted secondary trading is intended, explicitly document that tier gating applies only at the point of purchase and that NFT holders are not re-verified at distribution or maturity.

Evidences

Unrestricted NFT Transferability Bypasses Tier-Based Access Control After Purchase

Reproduce:

PoC Steps:

- Register a tier-1 gated product (requires Bronze SBT)
- Sanctioned address tries to invest directly → reverts `NotEligible`
- Qualified investor (has SBT) invests 500 USDT, gets NFT#1
- Qualified investor transfers NFT#1 to sanctioned address (no tier check on transfer)
- Admin triggers yield distribution → sanctioned address receives yield
- Admin triggers maturity → sanctioned address receives 500 USDT principal back
- Assertions confirm the sanctioned address profited despite never holding the SBT

PoC Code:

```

function test_SanctionedAddressReceivesYieldAndPrincipalViaNFTTransfer() publ
ic {
  // 1. Register a tier-gated product
  vm.prank(admin1);
  investment.registerProduct(
    Schema.RegisterProductArgs({
      productId: PRODUCT_1,
      offeringAmount: OFFERING_AMOUNT,
      minInvestment: MIN_INVESTMENT,
      offeringEndDate: OFFERING_END_DATE,
      maturityDate: MATURITY_DATE,
      expectedYield: EXPECTED_YIELD,
      operationStartDate: OPERATION_START_DATE,
      distributionStartDate: DISTRIBUTION_START_DATE,
      baseTokenURI: "https://example.com/1/",
      totalDistributionCount: 1,
      distributionInterval: 0,
      requiredTier: REQUIRED_TIER
    })
  );
}

```

```

    })
};

// 2. Verify sanctioned address CANNOT invest directly (no SBT)
usdt.mint(sanctionedAddr, MIN_INVESTMENT * 10);
vm.startPrank(sanctionedAddr);
usdt.approve(address(investment), 0);
usdt.approve(address(investment), MIN_INVESTMENT * 10);
vm.expectRevert(abi.encodeWithSelector(IInvestmentErrors.NotEligible.selector, REQUIRED_TIER));
investment.invest(PRODUCT_1, 1);
vm.stopPrank();

// 3. Qualified investor invests (has SBT, passes tier check)
uint256 investAmount = MIN_INVESTMENT * 5; // 500 USDT
usdt.mint(qualifiedInvestor, investAmount);
vm.startPrank(qualifiedInvestor);
usdt.approve(address(investment), 0);
usdt.approve(address(investment), investAmount);
investment.invest(PRODUCT_1, 5);
vm.stopPrank();

// 4. Get the NFT contract and token ID
Schema.Product memory p = investment.getProduct(PRODUCT_1);
IERC721 nft = IERC721(p.nftContract);
uint256 tokenId = 1;
assertEq(nft.ownerOf(tokenId), qualifiedInvestor);

// 5. Qualified investor transfers NFT to sanctioned address (standard ERC-721 transfer, no tier check)
vm.prank(qualifiedInvestor);
nft.transferFrom(qualifiedInvestor, sanctionedAddr, tokenId);
assertEq(nft.ownerOf(tokenId), sanctionedAddr);

// 6. Admin deposits yield funds and triggers distribution
uint256 depositAmount = investAmount * 2;
usdt.mint(admin1, depositAmount);
vm.startPrank(admin1);
usdt.approve(address(investment), 0);
usdt.approve(address(investment), depositAmount);
investment.deposit(PRODUCT_1, depositAmount);
vm.stopPrank();

vm.warp(DISTRIBUTION_START_DATE);

uint256 sanctionedBalanceBefore = usdt.balanceOf(sanctionedAddr);

vm.prank(admin1);
investment.distributeYield(PRODUCT_1);

```

```

uint256 sanctionedBalanceAfter = usdt.balanceOf(sanctionedAddr);

// 7. ASSERT: Sanctioned address received yield despite having NO SBT
assertGt(sanctionedBalanceAfter, sanctionedBalanceBefore, "Sanctioned address received yield without SBT");

// 8. Trigger maturity
vm.warp(MATURITY_DATE);

uint256 sanctionedBalanceBeforeMaturity = usdt.balanceOf(sanctionedAddr);

vm.prank(admin1);
investment.maturity(PRODUCT_1);

uint256 sanctionedBalanceAfterMaturity = usdt.balanceOf(sanctionedAddr);

// 9. ASSERT: Sanctioned address received principal back despite having NO SBT
assertGt(sanctionedBalanceAfterMaturity, sanctionedBalanceBeforeMaturity, "Sanctioned address received principal without SBT");
assertEq(
    sanctionedBalanceAfterMaturity - sanctionedBalanceBeforeMaturity,
    investAmount,
    "Full principal returned to sanctioned address"
);
}

```

Results:

```

Ran 1 test for test/PoC_TierBypassViaTransfer.t.sol:PoC_TierBypassViaTransfer
[PASS] test_SanctionedAddressReceivesYieldAndPrincipalViaNFTTransfer() (gas:
2685032)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 51.80ms (2.39ms
CPU time)

```

Files:

PoC_TierBypassViaTransfer.t.sol

[F-2026-16865](#) - Missing Zero-Address Validation in Initialization and Admin Management Leads to Permanent Misconfiguration - Low

Description:

The **Initialize** contract validates admin addresses against `address(0)` within the initialization loop but does not apply the same check to the `usdtAddress` or `safeMultisigWallet` parameters. The `initialize` function is protected by the `initializer` modifier from **Initializable**, which ensures it can only be called once.

```
function initialize(address[] calldata admins, address usdtAddress, address safeMultisigWallet)
    external
    initializer
{
    for (uint8 i = 0; i < admins.length; i++) {
        if (admins[i] == address(0)) revert IInvestmentErrors.InvalidAddress(
    );

    Storage.WhiteListsState().admins.push(admins[i]);
    // ...
    }

    // No zero-address check
    Storage.ConfigState().USDT_ADDRESS = usdtAddress;
    Storage.ConfigState().SAFE_MULTISIG_WALLET = safeMultisigWallet;
}
```

If either `usdtAddress` or `safeMultisigWallet` is mistakenly set to `address(0)`, the contract enters a permanently broken state. USDT transfer operations would fail on every call, and the safe multisig wallet reference would be irreversibly invalid. Because the `initializer` modifier prevents re-execution, there is no on-chain recovery path without redeployment.

Additionally, the `addAdmin` function in **ControlAdmin** does not validate the provided address against `address(0)`, despite `initialize` enforcing this check for the same admin array. This inconsistency allows a zero address to be appended to the admin whitelist after initialization, creating unexpected behavior in whitelist iteration and access control checks.

```
function addAdmin(address _admin) external onlyOwner {
    address[] memory admins = Storage.WhiteListsState().admins;
    for (uint256 i = 0; i < admins.length; i++) {
        if (admins[i] == _admin) {
            revert IInvestmentErrors.AlreadyExistsAdmin();
        }
    }
}
```



```

    }
    // No zero-address check
    Storage.WhiteListsState().admins.push(_admin);
    emit IInvestmentEvents.AdminAdded(_admin, msg.sender);
}

```

Assets:

- ControlAdmin.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Add zero-address validation for `usdtAddress` and `safeMultisigWallet` in `initialize`:

```

if (usdtAddress == address(0)) revert IInvestmentErrors.InvalidAddress();
if (safeMultisigWallet == address(0)) revert IInvestmentErrors.InvalidAddress();

```

Add zero-address validation in `addAdmin`:

```

if (_admin == address(0)) revert IInvestmentErrors.InvalidAddress();

```

[F-2026-16866](#) - Inconsistent Integer Size in Admin Array Iteration Leads to Denial of Service - Low

Description:

The protocol uses a whitelist-based access control system where an array of admin addresses is stored in **WhiteListsState**. Several functions iterate over this array to verify caller authorization or to populate it during initialization. The `addAdmin` function in **ControlAdmin**, which appends addresses to this array, uses a `uint256` loop counter and imposes no upper bound on the array length.

However, three separate locations iterate over the same admin array using a `uint8` loop counter:

```
function _isWhiteLists() internal view returns (bool) {
    address[] memory admins = Storage.WhiteListsState().admins;
    for (uint8 i = 0; i < admins.length; i++) {
        if (admins[i] == msg.sender) {
            return true;
        }
    }
    return false;
}
```

Since the contract uses Solidity version ^0.8.28, arithmetic overflow is checked by default. When the admin array length exceeds 255, the increment operation `i++` reverts on overflow when `i` reaches 255. This causes a permanent revert in `_isWhiteLists`, which gates all critical protocol operations: `deposit`, `withdraw`, `mintNFT`, `distributeYield`, `maturity`, `registerProduct`, and `setAllowedByTierAddress`. The same pattern in `setURI` within **InvestmentNFT** would similarly become non-functional.

The `initialize` function shares the same `uint8` loop counter, which effectively limits the initial admin array to at most 255 entries during deployment — exceeding this causes a revert before state is committed, preventing misconfiguration at that stage. The actual risk arises post-deployment: `addAdmin` in **ControlAdmin** uses a `uint256` counter and performs no length validation, allowing the owner to add a 256th admin without error. Once this occurs, all subsequent calls to `_isWhiteLists` revert on the `uint8` overflow, permanently disabling all whitelist-gated functions.

Assets:

- OnlyWhiteListsBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

- InvestmentNFT.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Enforce a consistent integer type across all admin array iterations. If the admin array is intended to remain small, add an explicit length check in `addAdmin` to cap the array at 255 entries and use `uint8` uniformly.
- If the array should grow beyond 255, change all loop counters to `uint256` instead.

F-2026-16870 - Restored Product Funding Does Not Clear isInsufficientBalance Flag, Causing Automation to Remain Stalled - Low

Description:

When a distribution or maturity settlement detects that the product pool is insufficient to cover the required payout, the flag `isInsufficientBalance` is set to `true` and the function returns early. **Automation** unconditionally skips every product carrying this flag:

```
if (product.isMaturity || product.isInsufficientBalance) {  
    continue;  
}
```

The `deposit` function allows an admin to restore funding by increasing `productPool`, but it never writes to `isInsufficientBalance`:

```
product.productPool += depositAmount;  
usdt.safeTransferFrom(msg.sender, address(this), depositAmount);
```

After a successful deposit, the product pool may cover the next settlement requirement, yet **Automation** continues to skip the product. Settlement can only resume when an admin manually calls `distributeYield` or `maturity`, neither of which gates on the flag and both of which clear it on successful completion.

The result is an undocumented operational dependency on manual lifecycle transactions after every temporary shortfall, delaying yield or principal settlement for NFT holders even when the pool is fully funded.

Assets:

- `DistributeYield.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Maturity.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Deposit.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Automation.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Clear the flag inside `deposit` so the recovery path is symmetric with the settlement functions that set it. Unconditional clearing is safe: if the pool is still short on the next upkeep, `distributeYield` or `maturity` will re-set the flag.

```
product.productPool += depositAmount;  
product.isInsufficientBalance = false;
```

Alternatively, modify `checkUpkeep` to re-evaluate flagged products by checking whether `productPool` now satisfies the settlement requirement instead of skipping them outright.

If manual retry after funding restoration is intentional, document it and emit a dedicated recovery event so operators can distinguish expected manual calls from unexpected automation gaps.

Evidences

Restored Product Funding Does Not Clear `isInsufficientBalance` Flag

Reproduce:

PoC Steps:

- Register a product and invest via `mintNFT` (fiat path) so `raisedAmount > 0` but `productPool` stays 0.
- Warp to distribution date. Call `distributeYield` - pool shortfall triggers `isInsufficientBalance = true`, distribution does not advance.
- Admin deposits 2x the raised amount via `deposit`, fully funding the pool.
- Assert: `productPool > 0` yet `isInsufficientBalance` is still `true` - `deposit` never clears the flag.
- Assert: `checkUpkeep` returns `false` - Automation skips the product despite sufficient funds.

PoC Code:

```
function test_InsufficientBalanceFlagNotCleared() public {  
    vm.prank(admin1);  
    investment.registerProduct(Schema.RegisterProductArgs({
```

```

        productId: PRODUCT_1,
        offeringAmount: OFFERING_AMOUNT,
        minInvestment: MIN_INVESTMENT,
        offeringEndDate: OFFERING_END_DATE,
        maturityDate: MATURITY_DATE,
        expectedYield: EXPECTED_YIELD,
        operationStartDate: OPERATION_START_DATE,
        distributionStartDate: MATURITY_DATE,
        baseTokenURI: "https://example.com/1/",
        totalDistributionCount: 1,
        distributionInterval: 0,
        requiredTier: 0
    }));

    // Invest via mintNFT (fiat path) -- productPool stays 0
    vm.prank(admin1);
    investment.mintNFT(PRODUCT_1, 4, investor1);

    Schema.Product memory p1 = investment.getProduct(PRODUCT_1);
    assertGt(p1.raisedAmount, 0);
    assertEq(p1.productPool, 0, "pool empty after fiat mint");

    vm.warp(MATURITY_DATE);

    // distributeYield detects shortfall and sets the flag
    vm.prank(admin1);
    investment.distributeYield(PRODUCT_1);

    p1 = investment.getProduct(PRODUCT_1);
    assertTrue(p1.isInsufficientBalance, "flag set after shortfall");
    assertEq(p1.distributedCount, 0, "distribution did not advance");

    // Admin restores funding with more than enough to cover yield + principal
    uint256 bigDeposit = p1.raisedAmount * 2;
    usdt.mint(admin1, bigDeposit);
    vm.startPrank(admin1);
    usdt.approve(address(investment), bigDeposit);
    investment.deposit(PRODUCT_1, bigDeposit);
    vm.stopPrank();

    p1 = investment.getProduct(PRODUCT_1);
    assertGt(p1.productPool, 0, "pool is now funded");
    assertTrue(p1.isInsufficientBalance, "flag NOT cleared by deposit");

    // Automation skips the product despite sufficient funds
    (bool needed,) = automation.checkUpkeep(abi.encode(0));
    assertFalse(needed, "checkUpkeep returns false - funded product stuck due

```

```
to stale flag");  
}
```

Results:

```
Ran 1 test for test/PoC_InsufficientBalanceFlagStale.t.sol:PoC_InsufficientBalanceFlagStale  
[PASS] test_InsufficientBalanceFlagNotCleared() (gas: 2508868)  
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 50.79ms (2.32ms CPU time)
```

Files:

PoC_InsufficientBalanceFlagStale.t.sol

F-2026-16949 - Missing Constraint Between Distribution Schedule End and Maturity Date Leads to Unexpected Locking Schedule - Low

Description:

The **RegisterProduct** contract validates temporal ordering between `offeringEndDate`, `operationStartDate`, `distributionStartDate`, and `maturityDate`. However, it does not enforce that the full distribution schedule completes on or before `maturityDate`.

The `registerProduct` function accepts `distributionStartDate`, `distributionInterval` (in months), and `totalDistributionCount` as independent parameters. The last scheduled distribution date is determined by adding `distributionInterval * (totalDistributionCount - 1)` months to `distributionStartDate`. No validation exists to ensure this computed end date falls on or before `maturityDate`.

```
if (args.maturityDate < args.offeringEndDate) {
    revert IInvestmentErrors.InvalidMaturityDate();
}
if (args.operationStartDate < args.offeringEndDate) {
    revert IInvestmentErrors.InvalidOperationStartDate();
}
if (args.distributionStartDate < args.operationStartDate) {
    revert IInvestmentErrors.InvalidDistributionStartDate();
}
// No check: lastDistributionDate <= maturityDate
```

The `maturity` function in **Maturity** requires that `distributedCount` is greater than or equal to `totalDistributionCount` before processing principal returns. Each call to `distributeYield` is time-gated by `nextDistributionDate`, which is computed from the distribution schedule using calendar-month arithmetic in **DistributionDateLib**. If the schedule extends past `maturityDate`, `maturity` reverts with `BeforeDistributionCompleted` at the disclosed maturity date and continues reverting until all scheduled distributions have been executed.

Product registration is a one-shot operation per `productId` with no setter for `maturityDate` or the distribution schedule parameters. Once a misconfigured product is registered, there is no on-chain recovery path. Investor principal remains locked for the full duration of the schedule overshoot, which can span months to years depending on the parameter mismatch.

Assets:

- RegisterProduct.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status: Pending Fix

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add a validation check in `registerProduct` that computes the last distribution date and reverts if it exceeds `maturityDate`:

```
uint256 lastDistributionDate = DistributionDateLib.calculateNextDistributionDate(
    args.distributionStartDate,
    args.distributionInterval,
    args.totalDistributionCount - 1,
    DistributionDateLib.isMonthEndDate(args.distributionStartDate)
);
if (lastDistributionDate > args.maturityDate) {
    revert IInvestmentErrors.InvalidMaturityDate();
}
```

F-2026-17029 - Flat Admin Whitelist Without Role Separation Grants Full Operational Power to Every Admin Key - Low

Description:

The protocol implements a single `onlyWhiteLists` modifier that gates all privileged operations through one flat admin array. No role-based access control separates distinct operational concerns:

```
function _isWhiteLists() internal view returns (bool) {
    address[] memory admins = Storage.WhiteListsState().admins;
    for (uint8 i = 0; i < admins.length; i++) {
        if (admins[i] == msg.sender) {
            return true;
        }
    }
    return false;
}
```

Every address in the admin list holds identical permissions across ten critical functions: `registerProduct`, `mintNFT`, `deposit`, `withdraw`, `distributeYield`, `maturity`, `setAllowedByTierAddress`, `setAllowedByTierId`, and the batch settlement paths. A single compromised admin key (through social engineering, phishing, or private key exposure) grants an attacker the ability to drain all product pools via `withdraw`, mint fraudulent investment positions via `mintNFT`, and modify tier access lists to block legitimate investors or whitelist malicious addresses.

Assets:

- OnlyWhiteListsBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- ControlAdmin.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Medium

Severity: Low

Recommendations

Remediation:

Extend the existing `addAdmin` function to accept a role bitmask, keeping the multisig as the super-admin that assigns scoped permissions:

```
// Storage: replace address[] admins with a role mapping
mapping(address => uint8) public adminRoles;

uint8 constant ROLE_PRODUCT_MANAGER = 1 << 0; // registerProduct, setTier
uint8 constant ROLE_TREASURY        = 1 << 1; // withdraw, deposit
uint8 constant ROLE_SETTLEMENT      = 1 << 2; // distributeYield, maturity
uint8 constant ROLE_MINTER          = 1 << 3; // mintNFT

function addAdmin(address _admin, uint8 _roles) external onlyOwner {
    adminRoles[_admin] = _roles;
}

modifier onlyRole(uint8 requiredRole) {
    if ((adminRoles[msg.sender] & requiredRole) == 0) revert NotAdmin();
    _;
}
```

Assign the Automation contract only `ROLE_SETTLEMENT` so its permission surface is limited to distribution and maturity.

Assign product managers `ROLE_PRODUCT_MANAGER` only, so that a compromised product-manager key cannot drain pools. Consider splitting treasury into `ROLE_DEPOSIT` (routine, assigned to operators) and restricting `withdraw` to `onlyOwner` (multisig) since withdrawals move investor capital out of the contract and occur less frequently than deposits. The multisig retains super-admin authority through `onlyOwner` on `addAdmin`/`deleteAdmin` and `withdraw`, preserving the existing trust hierarchy while limiting the blast radius of any single key compromise.

F-2026-16872 - Missing Duplicate Check in initialize Allows Redundant Admin Entries - Info

Description: The `initialize` function pushes each admin address to storage without checking for duplicates within the provided array:

```
for (uint8 i = 0; i < admins.length; i++) {  
    if (admins[i] == address(0)) revert IInvestmentErrors.InvalidAddress();  
    Storage.WhiteListsState().admins.push(admins[i]);  
}
```

In **ControlAdmin**, the `addAdmin` function enforces uniqueness by reverting on duplicates. This inconsistency means the initial admin list can contain the same address multiple times. The `deleteAdmin` function uses swap-and-pop, which removes only the first occurrence. If `admins = [Alice, Bob, Alice]` is passed at initialization, calling `deleteAdmin(Alice)` once leaves Alice still present in the array. A second deletion call is required for complete removal. The duplicate entries also inflate gas cost for every `_isWhiteLists` iteration across all admin-gated functions.

Assets:

- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status: Pending Fix

Classification

Impact Rate: 2/5
Likelihood Rate: 1/5
Exploitability: Dependent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Add a duplicate check in the initialization loop, mirroring the validation already present in `addAdmin`:

```
for (uint256 i = 0; i < admins.length; i++) {  
    if (admins[i] == address(0)) revert IInvestmentErrors.InvalidAddress();
```

```
for (uint256 j = 0; j < i; j++) {  
    if (admins[j] == admins[i]) revert IInvestmentErrors.AlreadyExistsAdmin();  
}  
Storage.WhiteListsState().admins.push(admins[i]);  
}
```

F-2026-16938 - Floating Pragma, Unpinned and Deprecated

Dependencies - Info

Description:

All contracts in the project use a floating pragma directive (`^0.8.28`). A floating pragma allows compilation with any compatible compiler version within the specified range, which means contracts may be compiled and deployed with a compiler version that has not been thoroughly tested against the codebase. Different compiler versions can introduce subtle behavioral changes or bugs that may affect contract execution in production.

Additionally, dependency libraries are included as git submodules without pinned commit hashes or version tags in the project configuration. Unpinned dependencies can silently introduce breaking changes or vulnerabilities when updated, as there is no guarantee that a future state of the dependency remains compatible or secure.

The project uses `smartcontractkit/chainlink-brownie-contracts` as a dependency, which has been officially deprecated by Chainlink. The repository README explicitly warns against using it for new projects and directs users to migrate to `smartcontractkit/chainlink-evm` as the canonical source for Chainlink Solidity contracts.

Assets:

- Invest.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Getter.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- DistributeYield.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- RegisterProduct.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Maturity.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Deposit.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Withdraw.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- MintNFT.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- SetTier.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- OnlyWhiteListsBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- ControlAdmin.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- OnlyOwnerBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Schema.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Storage.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

- `Investment.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentFunctions.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentErrors.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentEvents.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentFacade.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `CalculateYieldLib.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `DistributionDateLib.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `PurchasePermissionLib.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentNFT.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Automation.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentNFT.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `IAutomation.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `InvestmentDeployer.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `DeployInvestment.s.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `DeployAutomation.s.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Lock the Solidity pragma to a specific version across all contracts, for example change `pragma solidity ^0.8.28;` to `pragma solidity 0.8.28;`.
- Pin all git submodule dependencies to specific commit hashes or release tags in the project configuration.
- Migrate from the deprecated `smartcontractkit/chainlink-brownie-contracts` to `smartcontractkit/chainlink-evm` pinned to a specific release tag.
- Update `foundry.toml` remapping accordingly.

[F-2026-16945](#) - Missing Chain-Specific Scoping for Forwarder Address Environment Variable Risks Cross-Chain Misconfiguration - Info

Description:

The deployment scripts read some addresses using chain-scoped environment variables (`INVESTMENT_PROXY_ADDR_{chainId}`) but read others from non-scoped variables. In `DeployAutomation.s.sol` , the Chainlink forwarder address is non-scoped:

```
address investment = vm.envAddress(_chainIdConcat("INVESTMENT_PROXY_ADDR"));
address forwarder = vm.envAddress("FORWARDER_ADDR");
```

The same pattern exists in `DeployInvestment.s.sol` , where `USDT_ADDR` and `SAFE_MULTISIG_WALLET_ADDR` are also non-scoped:

```
address usdtAddress = vm.envAddress("USDT_ADDR");
address safeMultisigWallet = vm.envAddress("SAFE_MULTISIG_WALLET_ADDR");
```

All three addresses differ between chains (USDT has different contract addresses on Ethereum vs Polygon, the safe multisig wallet may differ per network, and Chainlink forwarder addresses are chain-specific).

A multi-chain deployment using a shared `.env` file may configure contracts with addresses belonging to a different network. The forwarder address is immutable (set in the constructor), and USDT and safe multisig wallet addresses are set in `initialize` (which can only be called once), requiring full redeployment to correct any misconfiguration.

Assets:

- Automation.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- DeployInvestment.s.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- DeployAutomation.s.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Scope all chain-specific environment variables by chain ID, consistent with the proxy address pattern:

```
address forwarder = vm.envAddress(_chainIdConcat("FORWARDER_ADDR_"));
address usdtAddress = vm.envAddress(_chainIdConcat("USDT_ADDR_"));
address safeMultisigWallet = vm.envAddress(_chainIdConcat("SAFE_MULTISIG_WALLET_ADDR_"));
```

[F-2026-16946](#) - Duplicate Implementation Deployment for SetTier Contract - Info

Description: The deployer library creates two separate instances of the **SetTier** contract for two different function selectors, while other contracts such as **ControlAdmin** and **Getter** deploy a single instance and register multiple selectors against it:

```
mc.use("setAllowedByTierAddress", SetTier.setAllowedByTierAddress.selector, address(new SetTier()));  
mc.use("setAllowedByTierId", SetTier.setAllowedByTierId.selector, address(new SetTier()));
```

Both instances are functionally identical because **SetTier** is stateless (all state is accessed through ERC-7201 proxy storage).

Assets:

- InvestmentDeployer.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status: Pending Fix

Classification

Impact Rate: 1/5
Likelihood Rate: 5/5
Exploitability: Dependent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Deploy a single **SetTier** instance and register both selectors against it:

```
address setTier = address(new SetTier());  
mc.use("setAllowedByTierAddress", SetTier.setAllowedByTierAddress.selector, setTier);  
mc.use("setAllowedByTierId", SetTier.setAllowedByTierId.selector, setTier);
```

F-2026-16947 - Unnecessary Custom String Parser for Admin Addresses Can Be Replaced With Native Foundry Cheatcode - Info

Description: The deployment script implements three custom helper functions to parse a comma-separated admin address string from an environment variable:

```
function _getAdminAddresses(string memory adminsStr) internal pure returns (address[] memory) {
    string[] memory adminsArray = _splitString(adminsStr, ",");
    address[] memory admins = new address[](adminsArray.length);
    for (uint256 i = 0; i < adminsArray.length; i++) {
        admins[i] = vm.parseAddress(adminsArray[i]);
    }
    return admins;
}
```

The manual `_splitString` and `_substring` helpers do not handle common formatting variations (trailing commas, double commas, whitespace after delimiters), causing unhelpful reverts from `vm.parseAddress` on malformed input.

Foundry natively supports reading typed arrays from environment variables via `vm.envAddress(key, delimiter)`, which handles parsing, validation, and error reporting internally. The entire custom parser is unnecessary.

Assets:

- DeployInvestment.s.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status: Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Replace the `_getAdminAddresses`, `_splitString`, and `_substring` functions with a single Foundry cheatcode call in `run()`:

```
address[] memory admins = vm.envAddress("ADMINS_ADDR", "");
```

F-2026-16955 - Missing Tier Registry Validation In Product Registration Can Temporarily Block Investment - Info

Description:

The **RegisterProduct** contract validates every field of the product registration arguments — offering amount, minimum investment, dates, yield, distribution count, and interval — but applies no validation to `requiredTier`. This `uint8` parameter accepts any value from 0 to 255 and is stored directly into the product struct.

When a non-zero `requiredTier` references a tier that has no entries in the tier registry, `hasPurchasePermission` in **PurchasePermissionLib** iterates over an empty `allowedByTierAddress` array and returns `false` for every caller. Both `invest` and `mintNFT` revert with `NotEligible` in this case, blocking all investment into the product until the tier is configured.

```
function hasPurchasePermission(address user, uint8 requiredTier) internal view
w returns (bool) {
    if (requiredTier == NONE) {
        return true;
    }

    address[] storage sbts = Storage.TierRegistryState().allowedByTierAddress
[requiredTier];
    uint256 len = sbts.length;
    for (uint256 i = 0; i < len;) {
        // ...
    }
    return false;
}
```

No function exists to update a product's `requiredTier` after registration. However, the tier registry itself is fully mutable: an admin can call `setAllowedByTierAddress` to populate any tier at any time, restoring the product to a functional state. The product, its NFT contract, and its `productId` are not permanently lost. This limits the impact to a temporary denial of service caused by admin misconfiguration, recoverable without redeployment.

Assets:

- RegisterProduct.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- PurchasePermissionLib.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Validate that the tier registry is populated when a non-zero `requiredTier` is supplied during product registration:

```
if (args.requiredTier != 0 &&  
    Storage.TierRegistryState().allowedByTierAddress[args.requiredTier].length  
    == 0) {  
    revert IInvestmentErrors.TierNotConfigured(args.requiredTier);  
}
```

Consider adding a function to update a product's `requiredTier` post-registration as an additional administrative recovery path.

[F-2026-16956](#) - Missing Constraints In Admin Deletion Allow Owner To Temporarily Block Operations - Info

Description:

The **ControlAdmin** contract exposes `deleteAdmin` as an `onlyOwner` function that removes an address from the `Storage.WhiteListsState().admins` array using a swap-and-pop pattern. The function performs no structural validation beyond confirming the target address exists in the array.

```
function deleteAdmin(address _admin) external onlyOwner {
    address[] storage admins = Storage.WhiteListsState().admins;
    bool found = false;
    uint256 length = admins.length;

    for (uint256 i = 0; i < length; i++) {
        if (admins[i] == _admin) {
            found = true;
            admins[i] = admins[length - 1];
            admins.pop();
            break;
        }
    }

    if (!found) {
        revert IInvestmentErrors.NotExistsAdmin();
    }

    emit IInvestmentEvents.AdminRemoved(_admin, msg.sender);
}
```

Several safety constraints expected for an admin registry of this scope are absent. The function does not reject removal of the final admin entry, allowing the array to reach length zero. Once the admin set is empty, every function gated by the `onlyWhiteLists` modifier reverts with `NotAdmin`. This includes `registerProduct`, `setAllowedByTierAddress`, `setAllowedByTierId`, `deposit`, `withdraw`, `distributeYield`, `maturity`, and `mintNFT`. The protocol becomes unable to register new products, configure tier eligibility, accept deposits from operations, distribute yield, process maturity, or mint NFTs to investors until the owner re-adds at least one admin.

The function also does not differentiate between operational admins and system-critical admin addresses such as the **Automation** contract. Any address bound into automation, oracle, or keeper integrations can be removed by the same call path used for routine admin rotation, with no separate registry or marker that prevents accidental removal of infrastructure addresses. Because the `_admin` argument is supplied by the

caller and matched only by direct equality against the array, an off-by-one in the input or a stale address provided by tooling can silently remove the wrong entry without producing an error.

These gaps do not expose the protocol to attacker-driven exploitation because `deleteAdmin` is `onlyOwner` and the multisig holds the only viable call path. They do, however, allow a single owner transaction to halt all privileged operations and silently break Chainlink Automation flows. Recovery is possible by calling `addAdmin` with the appropriate addresses, but no on-chain mechanism prevents or warns against the misconfiguration before it occurs.

Assets:

- ControlAdmin.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Reject removal of the last admin entry:

```
if (admins.length == 1) {  
    revert IInvestmentErrors.CannotRemoveLastAdmin();  
}
```

Consider tracking system-critical addresses such as the **Automation** contract in a dedicated config slot (for example `Storage.ConfigState().AUTOMATION_CONTRACT`) and reject deletion of any address present in that registry from `deleteAdmin`.

[F-2026-17004](#) - setAllowedByTierAddress Does Not Filter Zero-Address or Duplicate Entries - Info

Description: In **SetTier**, the `setAllowedByTierAddress` function copies the calldata array into storage without filtering:

```
Storage.TierRegistryState().allowedByTierAddress[tier] = sbts;
```

Zero-address entries and duplicates are accepted. Zero entries do not break correctness because `_hasPermissionByContract` returns `false` for `address(0)` and codeless addresses, but they inflate gas costs for every subsequent `invest` and `mintNFT` call that iterates the tier's SBT list. Duplicate entries cause the same SBT to be probed multiple times per investment call.

Assets:

- SetTier.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status: Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add a validation loop before the storage assignment:

```
for (uint256 i = 0; i < sbts.length; ++i) {
    if (sbts[i] == address(0) || sbts[i].code.length == 0) {
        revert IInvestmentErrors.TierSbtNotContract(sbts[i]);
    }
    for (uint256 j = 0; j < i; ++j) {
        if (sbts[j] == sbts[i]) revert IInvestmentErrors.DuplicateEntry();
    }
}
```

```
}  
}
```

F-2026-17008 - Dead owner != address(0) Check in distributeYield and maturity - Info

Description:

In both **DistributeYield** and **Maturity**, the lifecycle loops check `if (nftInfos[i].owner != address(0))` before processing each NFT. However, the upstream `getNFTInfos` function in **InvestmentNFT** calls `ownerOf(tokenId)` for every entry, and OpenZeppelin v5's `ownerOf` invokes `_requireOwned`, which reverts with `ERC721NonexistentToken` for any non-existent or burned token:

```
function _requireOwned(uint256 tokenId) internal view returns (address) {
    address owner = _ownerOf(tokenId);
    if (owner == address(0)) {
        revert ERC721NonexistentToken(tokenId);
    }
    return owner;
}
```

Every entry returned in the array is guaranteed to have a non-zero owner; `getNFTInfos` itself would revert before returning an entry with `address(0)`. The downstream branch can never evaluate to `false`, making it dead code.

Assets:

- `DistributeYield.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Maturity.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Remove the `owner != address(0)` guard from both **DistributeYield** and **Maturity**.

F-2026-17010 - Initialize Implementation Contract Not Locked with `_disableInitializers` Allows Direct Initialization - Info

Description:

The **Initialize** contract inherits `Initializable` but does not call `_disableInitializers` in a constructor. Under the ERC-7546 / MC framework, function contracts are deployed as standalone normal contracts at known addresses.

The MC proxy layer provides no equivalent protection against direct initialization of implementations. Any external caller can invoke `initialize` on the implementation contract directly, succeeding once because the implementation's own `_initialized` slot starts at zero.

This writes attacker-chosen admin addresses, USDT address, and Safe address to the implementation's own storage. While the proxy is unaffected (it has separate delegatecall storage), this degrades defense-in-depth, produces misleading on-chain reads for block explorers and monitoring tooling, and breaks OpenZeppelin's canonical guarantee that implementations cannot be initialized.

Assets:

- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Add a constructor that disables initializers per OpenZeppelin's standard recommendation:

```
contract Initialize is Initializable {
    constructor() {
        _disableInitializers();
    }
}
```

```
}

function initialize(
    address[] calldata admins,
    address usdtAddress,
    address safeMultisigWallet
) external initializer {
    // ...
}
}
```

[F-2026-17027](#) - Missing Event Emissions On State-Changing Functions Reduce Off-Chain Observability - Info

Description:

Several state-changing paths in the protocol mutate persistent storage without emitting events that fully capture the change. The omissions break the symmetry between on-chain mutations and the event log that off-chain consumers, monitoring systems, indexers, and the **Automation** keeper rely on for state reconstruction and alerting.

The `isInsufficientBalance` flag on each product has asymmetric event coverage. When the flag is raised, `InsufficientProductPoolForDistribution` or `InsufficientProductPoolForMaturity` is emitted to signal the stalled state. When the flag is cleared on a subsequent successful distribution or maturity, the assignment occurs silently with no corresponding clearance event.

```
// DistributeYield.sol
product.productPool -= _distributedYield;
product.isInsufficientBalance = false;
```

```
// Maturity.sol
product.productPool -= _returnedAmount;
product.isInsufficientBalance = false;
```

A monitoring system that subscribes to the insufficient-balance events to page operators when a product stalls cannot use the same event stream to confirm recovery. The system must poll product state to detect that automation has resumed, adding latency between recovery and resolution of the alert.

The `setURI` function in **InvestmentNFT** writes to `baseTokenURI` without emitting any event. This storage slot controls the metadata URI returned for every NFT in the collection. Off-chain indexers and marketplaces that track metadata changes have no log-based mechanism to detect a URI update and must resort to polling or manual notification.

```
function setURI(string memory newBaseTokenURI) external {
    address[] memory adminList = IInvestment(investmentContract).getAdminList();

    bool isAdmin = false;
    for (uint8 i = 0; i < adminList.length; i++) {
        if (adminList[i] == msg.sender) {
            isAdmin = true;
            break;
        }
    }
}
```

```

    }

    if (!isAdmin) {
        revert Ownable.OwnableUnauthorizedAccount(msg.sender);
    }

    baseTokenURI = newBaseTokenURI;
}

```

The `ProductRegistered` event emitted during product registration omits the `requiredTier` and `isMonthEnd` fields despite both values being written to storage in the same transaction. The event definition carries eleven parameters but excludes these two fields, making it impossible for indexers to reconstruct the full product configuration from event data alone.

```

emit IInvestmentEvents.ProductRegistered(
    args.productId,
    args.offeringAmount,
    args.minInvestment,
    args.offeringEndDate,
    args.maturityDate,
    args.expectedYield,
    args.operationStartDate,
    args.distributionStartDate,
    args.totalDistributionCount,
    args.distributionInterval,
    nftContract
);

```

The stored product struct includes `requiredTier: args.requiredTier` and `isMonthEnd: isMonthEnd`, neither of which appears in the event. Consumers that rely on the event log to build a product registry will miss tier gating configuration and distribution schedule type.

These omissions do not introduce a direct security vulnerability because the protocol can still function and the state is observable through view calls. They do degrade operational integrity: alerting pipelines that depend on log subscriptions become incomplete, metadata update tracking requires polling, and dashboards that track product lifecycle cannot fully reconstruct product configuration from events.

Assets:

- `DistributeYield.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `RegisterProduct.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- `Maturity.sol` [<https://github.com/DART-s-dev/rwa-investment-contracts>]

- IIInvestmentEvents.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- InvestmentNFT.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 4/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Define a clearance event for `isInsufficientBalance` and emit it in both **DistributeYield** and **Maturity** at the point where the flag is reset:

```
event ProductPoolRecovered(uint256 indexed productId);

product.isInsufficientBalance = false;
emit IIInvestmentEvents.ProductPoolRecovered(productId);
```

Emit an event from `setURI` in **InvestmentNFT** after writing the new URI:

```
event BaseTokenURIUpdated(string newBaseTokenURI);

baseTokenURI = newBaseTokenURI;
emit BaseTokenURIUpdated(newBaseTokenURI);
```

Add `requiredTier` and `isMonthEnd` parameters to the `ProductRegistered` event definition and include them in the emit statement.

F-2026-17028 - Test Contracts In Production Code - Info

Description:

Every production Solidity file under `src/` contains an inline test section after the production contract definition. The test section imports test-only dependencies such as `@mc-devkit/Flattened.sol`, `forge-std/Test.sol`, and mock contracts under `test/`, then defines one or more test contracts with `test_*`, `testFuzz_*`, and helper functions. The pattern is consistent across the investment bundle, utility libraries, and periphery contracts.

```
// production contract definition

// Testing
import {MCTest} from "@mc-devkit/Flattened.sol";

contract RegisterProductTest is MCTest {
    // setUp, test_*, testFuzz_* ...
}
```

This is not a security vulnerability in itself and the pattern appears to be intentional, likely a convention enforced by the MCDevKit framework used in this codebase. The finding is raised as a best-practice and maintainability concern.

Assets:

- Invest.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Getter.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- DistributeYield.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- RegisterProduct.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Maturity.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Deposit.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Withdraw.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- MintNFT.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- SetTier.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- OnlyWhiteListsBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- ControlAdmin.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- OnlyOwnerBase.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Initialize.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- CalculateYieldLib.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

- DistributionDateLib.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- PurchasePermissionLib.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]
- Automation.sol [<https://github.com/DART-s-dev/rwa-investment-contracts>]

Status:

Pending Fix

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Move all test contracts, test-only imports, mocks, and fuzz harnesses out of `src/` and into a dedicated `test/` tree organized to mirror the production layout (for example `test/investment/functions/onlyWhiteLists/RegisterProduct.t.sol`).
- Restrict `src/` to deployable production code and library code that is intentionally part of the deployment artifact set.
- Reconfigure the build profile so that `forge build` for deployment compiles only `src/`, while `forge test` continues to compile `src/` and `test/` together. Foundry supports this through `foundry.toml` profiles.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/DART-s-dev/rwa-investment-contracts
Commit	2448ddd484c4e06ba81e207ba8e1e95781ce9879
Whitepaper	N/A
Requirements	https://github.com/DART-s-dev/rwa-investment-contracts/tree/main/docs
Technical Requirements	https://github.com/DART-s-dev/rwa-investment-contracts/tree/main/docs

Asset	Type
Automation.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
CalculateYieldLib.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
ControlAdmin.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
DeployAutomation.s.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
DeployInvestment.s.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Deposit.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
DistributeYield.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
DistributionDateLib.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Getter.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IAutomation.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IIInvestment.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IIInvestmentErrors.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IIInvestmentEvents.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IIInvestmentFunctions.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
IIInvestmentNFT.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract

Asset	Type
Initialize.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Invest.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
InvestmentDeployer.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
InvestmentFacade.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
InvestmentNFT.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Maturity.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
MintNFT.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
OnlyOwnerBase.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
OnlyWhiteListsBase.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
PurchasePermissionLib.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
RegisterProduct.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Schema.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
SetTier.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Storage.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract
Withdraw.sol [https://github.com/DART-s-dev/rwa-investment-contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.